Established – 1961 Subject: OS DBMS

SEVA SADAN'S

R. K. TALREJA COLLEGE

OF

ARTS, SCIENCE & COMMERCE

ULHASNAGAR – 421 003

CERTIFICATE

This is to certify that Mr. Divya Subhash Chandra Tripathi of S.Y. Information Technology Roll No. 2542049 has satisfactorily completed the Open Source DataBase Management System Mini Project Database Normalization Case Study During the academic year 2025 – 2026, as a part of the practical requirement. The project work is found to be satisfactory and is approved for submission.

PROF. INCHARGE **HEAD OF DEPT**

_____ _____

ABSTRACT

This project presents a detailed case study on Database Normalization using MySQL. The main objective of this project is to understand and implement normalization techniques to reduce data redundancy and improve data integrity in a relational database system.

The study begins with an unnormalized table containing redundant employee, project, and manager information. The database is then systematically converted into First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). Each stage removes specific types of dependency and redundancy to achieve a well-structured database design.

The implementation is performed using MySQL Command Line Client. The final database structure consists of multiple related tables connected through primary and foreign keys, ensuring consistency and efficient data management.

This case study demonstrates the practical importance of normalization in real-world database design and management.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to our subject teacher for giving me the opportunity to work on the project titled “Database Normalization Case Study.” This project has helped me enhance my understanding of database concepts and normalization techniques.

I am thankful for the valuable guidance and support provided throughout the completion of this project. The knowledge gained during this project will be highly beneficial for my academic and professional growth.

I would also like to thank my classmates and friends for their support and encouragement.

INDEX

Sr. No.	Chapter Title	PAGE NO
1	Introduction	5
2	Problem Definition	6
3	Objectives of the Project	7
4	Scope of the Project	8
5	Requirement Specification	9
6	System Design	10
7	Database Design	11-14
8	UML Diagrams	15-18
9	SQL Implementation	19-21
10	System Testing and Result	22-25
11	Security, Backup and Recovery	26-27
12	Future Scope and Conclusion	28
13	References	28
14	Glossary	28
15	APPENDIX/SQL Code	29-30

1. INTRODUCTION

A database system is a structured method of storing, organizing, and retrieving data efficiently. In many organizations, large amounts of information such as employee details, project records, and skill information must be maintained properly. If data is not organized correctly, it leads to duplication, inconsistency, and errors.

In traditional systems, employee data is often stored in a single large table containing repeated information such as department names, project managers, and employee skills. This causes problems like data redundancy, update anomalies, insertion anomalies, and deletion anomalies. For example, if a manager's name changes, it must be updated in multiple rows, increasing the chance of inconsistency.

The **Database Normalization Case Study** project demonstrates how unorganized data can be structured using normalization rules in MySQL. Normalization is a systematic process of organizing data in a database to reduce redundancy and improve data integrity.

In this project, an unnormalized employee-project database is transformed step-by-step into structured tables by applying 1NF, 2NF, 3NF, and BCNF. Separate tables such as Employees, Projects, Skills, and relationship tables are created. Primary keys and foreign keys are used to maintain relationships and ensure consistency.

MySQL is selected because it is open-source, reliable, widely used, and suitable for academic as well as small-scale business applications. It supports SQL queries, constraints, transaction control, indexing, and security mechanisms.

This project provides practical exposure to database normalization concepts and demonstrates how proper database design improves storage efficiency, data accuracy, and query performance.

2. PROBLEM DEFINITION

Many organizations maintain employee and project data in a single flat table. This approach leads to several problems:

• **Data Redundancy** – Repeated department names, manager names, and project details

• **Data Inconsistency** – Updating data in one row but not in others

• **Update Anomaly** – Changing one value requires multiple updates

• **Insert Anomaly** – Cannot insert new project without employee data

• **Delete Anomaly** – Deleting an employee may remove project information

• **Multi-Valued Attributes** – Employees having multiple skills in one column

• **Slow Query Performance** – Large unstructured tables reduce efficiency

• **Increased Storage Usage** – Duplicate values consume more space

These issues highlight the need for a structured relational database using normalization techniques.

3. OBJECTIVES OF THE PROJECT

The main objectives of this project are:

• To convert unnormalized data into a structured relational schema
• To apply 1NF, 2NF, 3NF, and BCNF rules
• To remove redundancy and anomalies
• To create multiple related tables using primary and foreign keys
• To implement SQL queries for data manipulation
• To analyze query performance improvement
• To demonstrate storage optimization
• To gain practical experience with MySQL

4. SCOPE OF THE PROJECT

The scope of the **Database Normalization Case Study** includes the following areas:

• **Conversion of unnormalized data into structured tables** – The project converts a single large table into multiple related tables to remove duplication.

• **Application of 1NF, 2NF, 3NF, and BCNF** – Normalization rules
are applied step-by-step to organize data properly.

• **Creation of entities and relationships** – Tables like Employees, Projects, and Skills are created and connected using primary and foreign keys.

• **Implementation of SQL operations** – SQL commands such as INSERT, UPDATE, DELETE, and SELECT are used to manage data.

• **Use of constraints** – PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE constraints are applied to maintain data integrity.

• **Basic query optimization** – JOIN operations and simple indexing are used to improve query performance.
The project is limited to backend database design using MySQL and is mainly intended for academic learning and small-scale data management.

5. REQUIREMENT SPECIFICATION

a. Hardware Requirements

Component	Specification
Output Devices	Moniter/Laptop
Input Devices	Keyboard, Mouse
Processeor	Intel i3 or above
RAM	Minimum 4GB
Hard Disk	Minimum 20GB

b. Software Requirements

Software	Purpose
MySQL Server	Database creation and management
MySQL Workbench	Query execution interface
Windows OS	Platform
SQL	Query language

6. SYSTEM DESIGN

6.1 System Architecture

The **Database Normalization Case Study** follows a simple database-centered architecture. In this system, the user interacts directly with the database using SQL queries executed through MySQL Workbench. There is no frontend interface; the entire system operates at the backend level.

The core component of the system is MySQL Server. It is responsible for storing, managing, and processing all employee, project, and skill data. The server ensures that data is stored in normalized tables and maintains relationships using primary keys and foreign keys.

When a user executes an SQL command such as INSERT, UPDATE, DELETE, or SELECT, the query is sent to the MySQL Server. The server validates the query syntax, checks constraints, verifies relationships between tables, and processes the request. If the query violates any constraint (for example, duplicate primary key or invalid foreign key reference), the server rejects the operation to maintain data integrity.

The architecture ensures:

- Proper organization of data into multiple related tables
- Elimination of redundancy through normalization
- Enforcement of data integrity using constraints
- Secure access through user privileges
- Reliable execution of transactions using COMMIT and ROLLBACK

6.2 Data Flow

The data flow in the system is straightforward:

The user provides input through SQL commands. The MySQL Server processes the request, performs necessary validations, updates the database if required, and returns the result in tabular form.

7. DATABASE DESIGN

Each entity represents a separate real-world object and helps in organizing data in a normalized structure.

7.1 Entity Description

The database is divided into multiple entities to maintain a normalized structure. Each entity represents a real-world component of the employee-project management system and is connected using primary and foreign keys.

Employees – Stores employee details such as employee ID, name, and department. Each employee is uniquely identified by emp_id.

Projects – Contains information about projects such as project ID, project name, and start date. Each project is identified by proj_id.

Skills –Maintains a list of skills possessed by employees. Each skill has a unique skill_id.

EmployeeProjects –A relationship table that connects Employees and Projects. It manages the many-to-many relationship between them and may include working hours.

ProjectManagers –Stores information about which employee manages a particular project using foreign key references.
These entities together create a structured and normalized database design using MySQL**.**

7.2 Tables Structure

1. **Employee Table**

Attribute	Data Type	Constraint
emp_id	INT	Primary Key
name	VARCHAR(50)	NOT NULL
dept	VARCHAR(30)	NOT NULL

2. **Project Table**

Attribute	Data Type	Constraint
proj_id	INT	Primary Key
proj_name	VARCHAR(50)	NOT NULL
start_date	DATE	NOT NULL

3. **Skill Table**

Attribute	Data Type	Constraint
skill_id	INT	Primary Key
skill_name	VARCHAR(30)	UNIQUE, NOT NULL

4. **EmployeeProject Table**

Attribute	Data Type	Constraint
emp_id	INT	Foreign Key
proj_id	INT	Foreign Key
hours	INT	CHECK (hours > 0)

5. **ProjectManager Table**

Attribute	Data Type	Constraint
proj_id	INT	Foreign Key
mgr_id	INT	Foreign Key

7.3 Constraints Used

In this project, various constraints are applied in MySQL to maintain accuracy, consistency, and proper relationships between tables.

PRIMARY KEY – The PRIMARY KEY constraint uniquely identifies each record in a table. In this project, emp_id, proj_id, and skill_id are used as primary keys. This ensures that no duplicate records exist in the main entity tables.

FOREIGN KEY – The FOREIGN KEY constraint establishes relationships between tables. For example, emp_id and proj_id in the EmployeeProjects table reference the Employees and Projects tables. This ensures that only valid and existing records are linked, preventing orphan records.

NOT NULL – The NOT NULL constraint ensures that important fields such as employee name, department, project name, and skill name cannot be left empty. This guarantees that essential information is always stored.

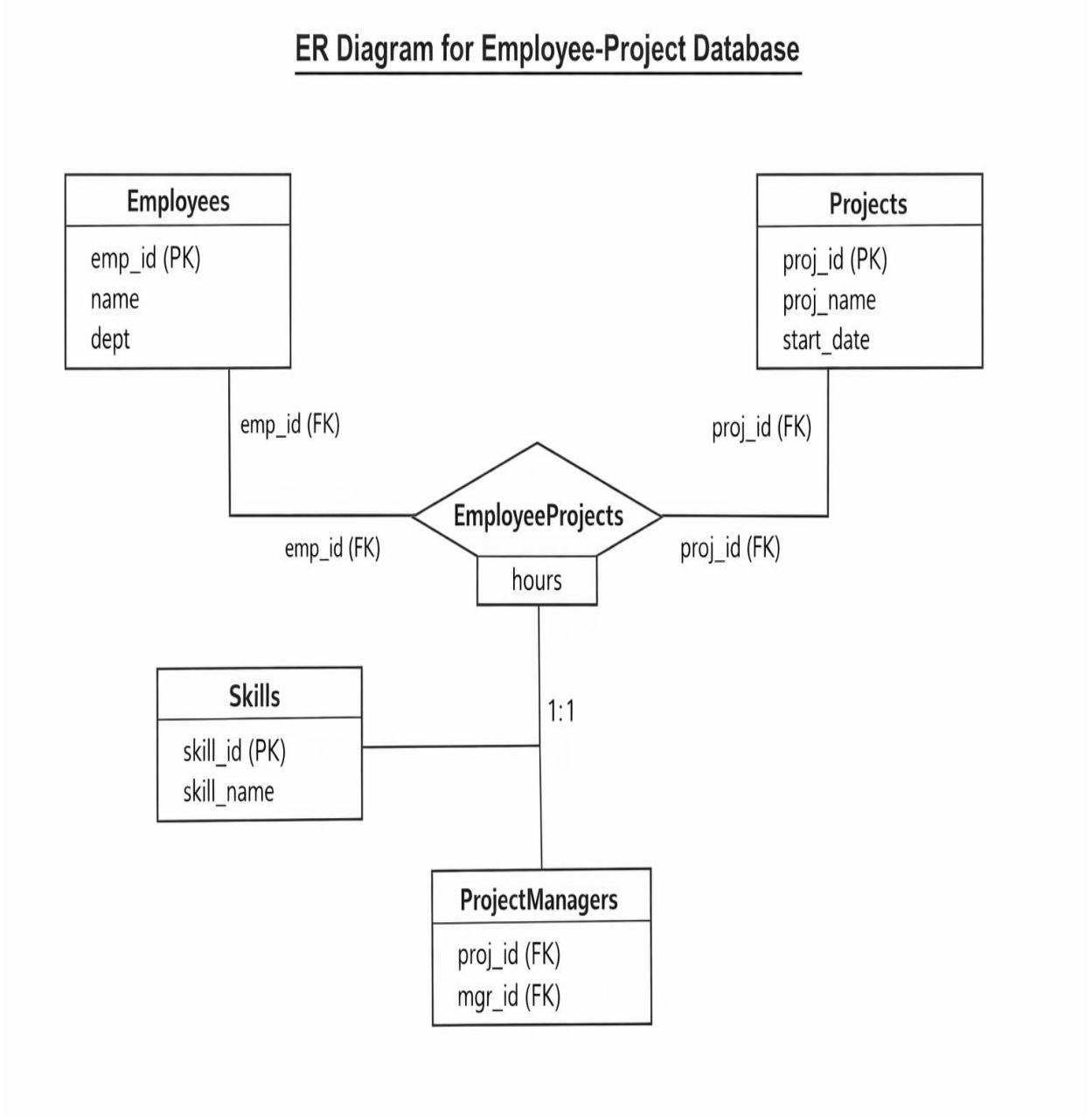
UNIQUE – The UNIQUE constraint prevents duplicate values in specific fields. In this project, skill_name is defined as UNIQUE to avoid storing the same skill multiple times.

CHECK (hours > 0) – The CHECK constraint is applied to the hours attribute in the EmployeeProjects table to ensure that working hours are always greater than zero. This prevents invalid or negative values from being entered into the database.

8. UML DIAGRAMS

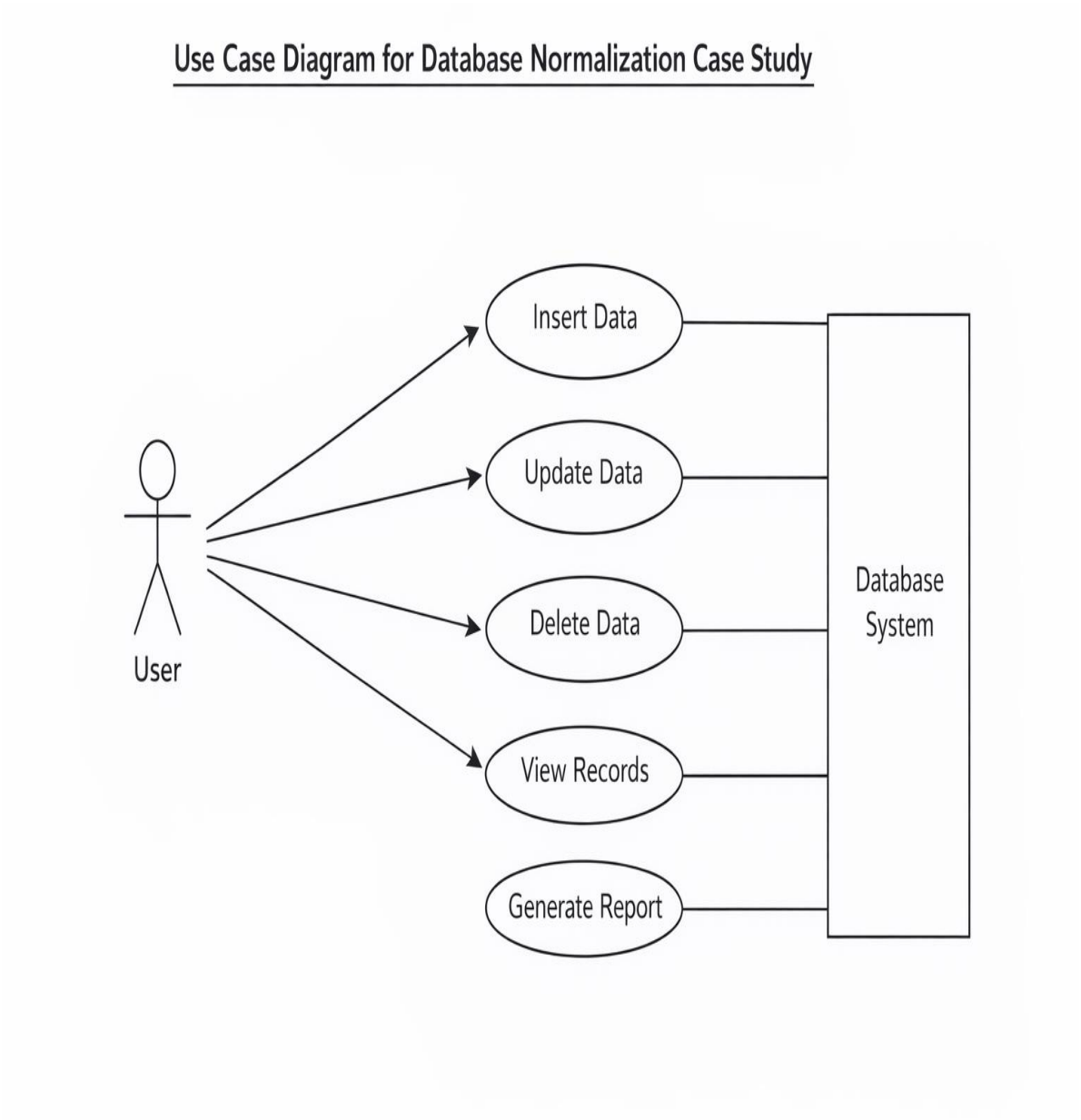
8.1. ER-Diagram

Shows relationships between Employees, Projects, and Skills.

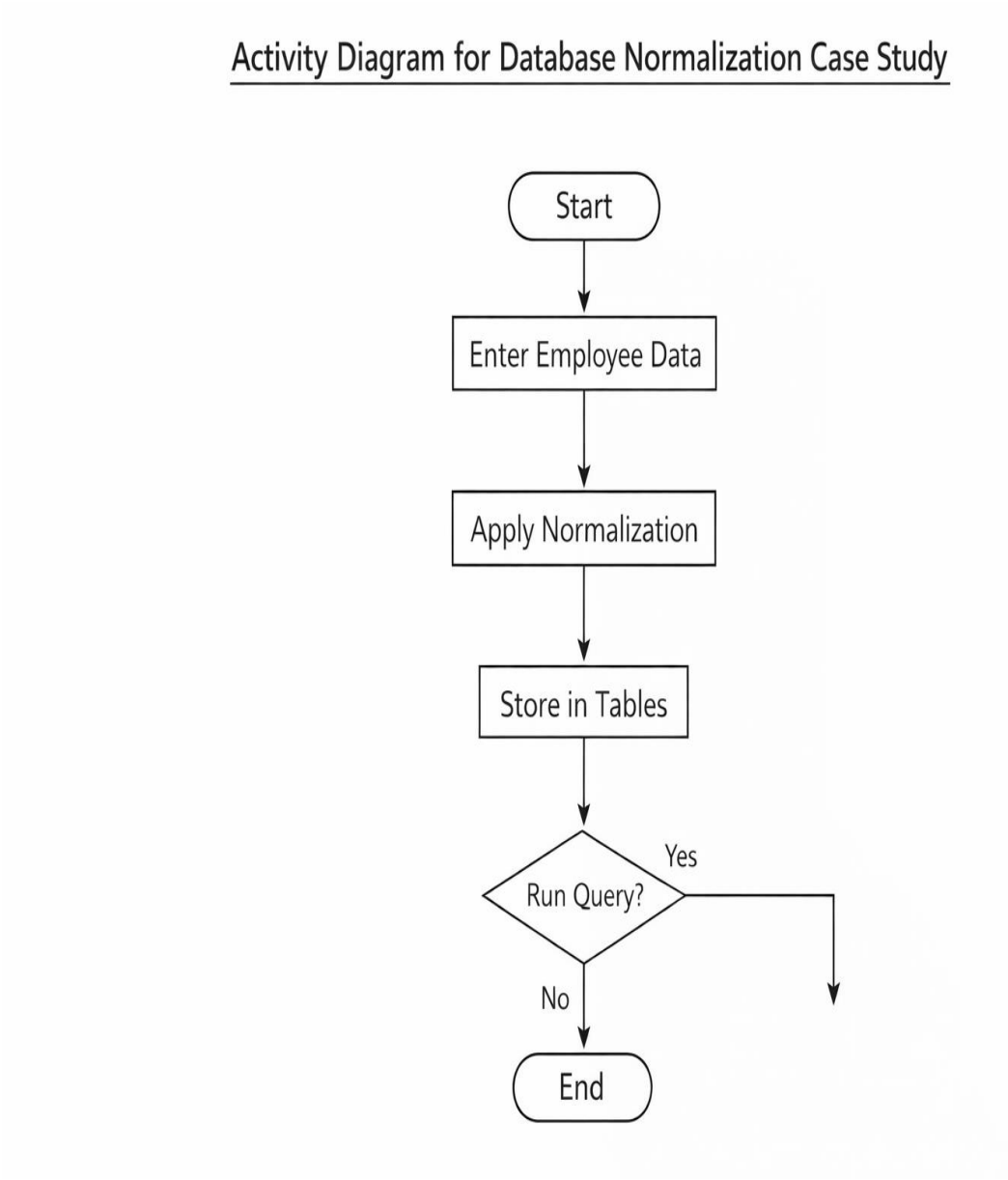


8.2. Use Case Diagram

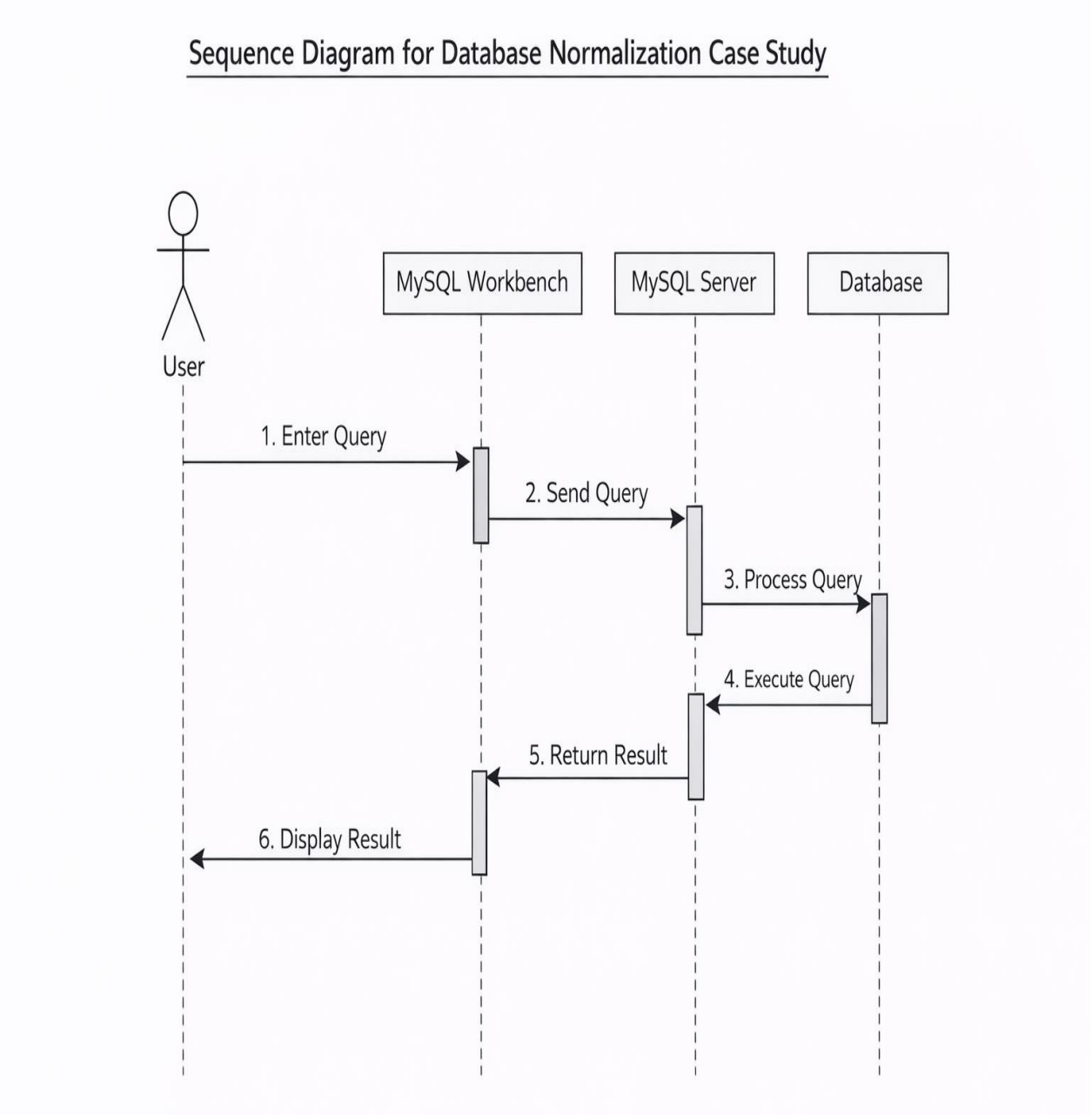
User performs: Insert, Update, Delete, View Reports.



8.3 Activity Diagram
Shows normalization process workflow.



8.4 Sequence Diagram
Shows query execution sequence between user and MySQL server.



9. SQL IMPLEMENTATION

The SQL implementation of the **Database Normalization Case Study** was performed using MySQL.
The implementation demonstrates how an unnormalized table is converted into a normalized database structure by applying 1NF, 2NF, and 3NF rules.

9.1. Creation of Unnormalized Table

Initially, all employee-related information was stored in a single table containing repeated and multi-valued data.

```
CREATE TABLE Employee_Unnormalized (
   emp_id INT,
   emp_name VARCHAR(100),
   department VARCHAR(100),
   project_name VARCHAR(150),
   manager_name VARCHAR(100),
   skills VARCHAR(200)
);
```

Sample Data Insertion

```
INSERT INTO Employee_Unnormalized VALUES
(1, 'Amit', 'IT', 'Website Development', 'Rahul', 'SQL, Python'),
(1, 'Amit', 'IT', 'App Development', 'Rahul', 'SQL, Python'),
(2, 'Priya', 'HR', 'Recruitment System', 'Suman', 'Communication, Excel');
```

This structure leads to:

- Data redundancy
- Update anomaly
- Multi-valued attributes

9.2. Conversion to First Normal Form (1NF)

To achieve 1NF, repeating groups and multi-valued attributes were removed.

```
CREATE TABLE Employees_1NF (
   emp_id INT,
   emp_name VARCHAR(100),
   department VARCHAR(100),
   project_name VARCHAR(150),
   manager_name VARCHAR(100),
   skill VARCHAR(100)
);
```

Each skill is stored in a separate row to remove multi-valued attributes.

9.3. Conversion to Second Normal Form (2NF)

To achieve 2NF, partial dependencies were removed by separating employee and project data into different tables.

```
CREATE TABLE Employees (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(100),
   department VARCHAR(100)
);

CREATE TABLE Projects (
   proj_id INT PRIMARY KEY AUTO_INCREMENT,
   project_name VARCHAR(150),
   manager_name VARCHAR(100)
);
```

A relationship table was created:

```
CREATE TABLE EmployeeProjects (
   emp_id INT,
   proj_id INT,
   PRIMARY KEY (emp_id, proj_id),
   FOREIGN KEY (emp_id) REFERENCES Employees(emp_id),
   FOREIGN KEY (proj_id) REFERENCES Projects(proj_id)
);
```

9.4. Conversion to Third Normal Form (3NF)

To achieve 3NF, transitive dependencies were removed. Manager information was separated into a new table.

```
CREATE TABLE Managers (
   mgr_id INT PRIMARY KEY AUTO_INCREMENT,
   manager_name VARCHAR(100) UNIQUE
);
```

Projects table updated:

```
CREATE TABLE Projects_3NF (
   proj_id INT PRIMARY KEY AUTO_INCREMENT,
   project_name VARCHAR(150),
   mgr_id INT,
   FOREIGN KEY (mgr_id) REFERENCES Managers(mgr_id)
);
```

9.5. Final Normalized Structure

Final tables in 3NF:

- Employees
- Projects
- Managers
- Skills
- EmployeeProjects

This structure eliminates redundancy and prevents anomalies.

9.6. Data Retrieval Using JOIN

```
SELECT e.emp_name, p.project_name, m.manager_name
FROM Employees e
JOIN EmployeeProjects ep ON e.emp_id = ep.emp_id
JOIN Projects_3NF p ON ep.proj_id = p.proj_id
JOIN Managers m ON p.mgr_id = m.mgr_id;
```

9.7. Transaction Management

```
START TRANSACTION;
INSERT INTO Employees VALUES (3, 'Rohan', 'Finance');
COMMIT;
```

Transaction control ensures data consistency during multiple operations.

Result of SQL Implementation

After normalization:

• Redundancy reduced
• Update anomalies eliminated
• Data integrity maintained
• Query performance improved

10.SYSTEM TESTING AND RESULT

10.1 Database Creation and Use

Definition:

This test verifies whether the database is created successfully without errors and usable.

```
mysql> CREATE DATABASE NormalizationDB;
Query OK, 1 row affected (0.061 sec)

mysql> USE NormalizationDB;
Database changed
```

10.2 Unnormalized Table Testing

Definition:

This test checks the creation of an unnormalized table containing redundant data.

Expected Result:

Table created successfully.

```
mysql> desc Employee_Unnormalized;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| emp_id       | int          | YES  |     | NULL    |       |
| emp_name     | varchar(100) | YES  |     | NULL    |       |
| department   | varchar(100) | YES  |     | NULL    |       |
| project_name | varchar(150) | YES  |     | NULL    |       |
| manager_name | varchar(100) | YES  |     | NULL    |       |
| skills       | varchar(200) | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
6 rows in set (0.073 sec)
```

10.3 Insertion Testing (Redundant Data)

Definition:

This test inserts duplicate and multi-valued data to show redundancy.

Expected Result:

Data inserted with repetition (shows redundancy).

```
mysql> select * from Employee_Unnormalized;
+--------+----------+------------+---------------------+--------------+-------------+
| emp_id | emp_name | department | project_name        | manager_name | skills      |
+--------+----------+------------+---------------------+--------------+-------------+
|      1 | Amit     | IT         | Website Development | Rahul        | SQL, Python |
|      1 | Amit     | IT         | App Development     | Rahul        | SQL, Python |
+--------+----------+------------+---------------------+--------------+-------------+
2 rows in set (0.010 sec)
```

10.4 First Normal Form (1NF) Testing

Definition:
This test ensures removal of multi-valued attributes.
Expected Result:
Table created with atomic values.

```
mysql> desc Employees_1NF;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| emp_id       | int          | YES  |     | NULL    |       |
| emp_name     | varchar(100) | YES  |     | NULL    |       |
| department   | varchar(100) | YES  |     | NULL    |       |
| project_name | varchar(150) | YES  |     | NULL    |       |
| manager_name | varchar(100) | YES  |     | NULL    |       |
| skill        | varchar(100) | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
6 rows in set (0.039 sec)
```

10.5 Primary Key Constraint Testing

Definition:
This test verifies that duplicate primary keys are not allowed.
Expected Result:
Second insertion fails due to duplicate primary key.

```
mysql>
mysql> INSERT INTO Employees VALUES (1, 'Amit', 'IT');
Query OK, 1 row affected (0.048 sec)

mysql> INSERT INTO Employees VALUES (1, 'Rahul', 'HR');
ERROR 1062 (23000): Duplicate entry '1' for key 'employees.PRIMARY'
```

10.6 Foreign Key Constraint Testing

Definition:
This test ensures relationship integrity between tables.
Expected Result:
Insertion fails because emp_id 5 does not exist.

```
mysql>
mysql> INSERT INTO EmployeeProjects VALUES (5, 1);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`normalizationdb`.`employeeproject
s`, CONSTRAINT `employeeprojects_ibfk_1` FOREIGN KEY (`emp_id`) REFERENCES `employees` (`emp_id`))
```

10.7 Second Normal Form (2NF) Testing
Definition:
This test verifies removal of partial dependency by separating employee and project tables.
Expected Result:
Data inserted successfully showing proper relationship.

```
+----------+---------------------+
| emp_name | project_name        |
+----------+---------------------+
| Amit     | Website Development |
+----------+---------------------+
1 row in set (0.007 sec)
```

10.8 Third Normal Form (3NF) Testing
Definition:
This test verifies removal of transitive dependency by separating managers table.
Expected Result:
Managers stored separately ensuring no redundancy.

```
mysql> desc Managers;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| mgr_id       | int          | NO   | PRI | NULL    |       |
| manager_name | varchar(100) | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
2 rows in set (0.037 sec)
```

10.9 Data Retrieval Testing
Definition:
This test checks whether normalized tables correctly retrieve data using JOIN.
Expected Result:
Correct combined data displayed without redundancy.

```
mysql> SELECT e.emp_name, p.project_name
    -> FROM Employees e
    -> JOIN EmployeeProjects ep ON e.emp_id = ep.emp_id
    -> JOIN Projects p ON ep.proj_id = p.proj_id;
+----------+---------------------+
| emp_name | project_name        |
+----------+---------------------+
| Amit     | Website Development |
+----------+---------------------+
1 row in set (0.007 sec)
```

10.10 Final Result

Definition:

After applying normalization (1NF, 2NF, 3NF), the database structure becomes efficient and free from anomalies.

Result Observed:

- Data redundancy removed
- Update anomaly eliminated
- Referential integrity maintained
- Structured relational database achieved

```
mysql> SHOW TABLES;
+---------------------------+
| Tables_in_normalizationdb |
+---------------------------+
| employee_unnormalized     |
| employeeprojects          |
| employees                 |
| employees_1nf             |
| managers                  |
| projects                  |
+---------------------------+
6 rows in set (0.203 sec)
```

11.SECURITY, BACKUP AND RECOVERY

11.1 Database Security

Database security ensures that only authorized users can access and modify data.

User Authentication

Each user must log in using a valid username and password before accessing the database.

```
CREATE USER 'admin_user'@'localhost' IDENTIFIED BY 'password123';
```

Granting Privileges

Access rights are assigned according to user roles.

```
GRANT ALL PRIVILEGES ON NormalizationDB.*
TO 'admin_user'@'localhost';
```

This ensures:

- Only authorized users can modify normalized tables
- Unauthorized access is restricted
- Data integrity is maintained

Role-Based Access Control

Different users can have different permissions.

```
GRANT SELECT ON NormalizationDB.*
TO 'viewer_user'@'localhost';
```

Viewer users can only read data but cannot modify it.

11.2 Backup Strategy

Backup ensures that data can be restored in case of accidental deletion, corruption, or system crash.

Full Database Backup

Using command line:

```
mysqldump -u root -p NormalizationDB > normalization_backup.sql
```

This creates a complete backup file of the database.

Table-Level Backup

```
CREATE TABLE Employees_Backup AS
SELECT * FROM Employees;
```

This creates a backup copy of the Employees table.

11.3 Recovery Process

Recovery restores the database from backup files.

Restore Full Database

```
mysql -u root -p NormalizationDB < normalization_backup.sql
```

Restore From Backup Table

```
INSERT INTO Employees
SELECT * FROM Employees_Backup;
```

11.4 Transaction Control for Recovery

Transactions help in maintaining consistency during multiple operations.

```
START TRANSACTION;
INSERT INTO Employees VALUES (5, 'Rohan', 'Finance');
ROLLBACK;
```

If any error occurs, ROLLBACK restores the previous stable state.

Result

After implementing security, backup, and recovery mechanisms:

- Database access is restricted to authorized users
- Data integrity is maintained
- Backup copies are available
- Database can be restored in case of failure
- Normalized structure remains consistent and secure

12.FUTURE SCOPE AND CONCLUSION

The Database Normalization Case Study successfully demonstrates how applying normalization improves data integrity, reduces redundancy, and enhances database efficiency.

12.1 Future Scope

• The database can be extended to higher normal forms like BCNF.
• A front-end application can be developed for user interaction.
• The system can be deployed on cloud platforms.
• Automated normalization tools can be implemented in the future.

12.2 Conclusion

The Database Normalization Case Study shows how converting an unnormalized table into 1NF, 2NF, and 3NF removes redundancy and prevents anomalies.
After normalization:
• Data integrity is maintained
• Redundancy is reduced
• Database structure becomes efficient and organized

Thus, normalization is essential for designing a reliable relational database system.

13.REFERENCES

1. MySQL Official Documentation, Oracle Corporation.
2. Prescribed Database Management System (DBMS) textbooks.
3. Online learning resources related to SQL and MySQL.

14.GLOSSARY

DBMS – Database Management System
Software used to store, manage, and retrieve data efficiently in the form of databases.
SQL – Structured Query Language
A standard language used to create, insert, update, delete, and retrieve data from a database
Normalization – Process of organizing data
Primary Key – Unique identifier
A unique identifier for each record in a table that does not allow duplicate or NULL values.
Foreign Key – Relationship key
A field in a table that creates a relationship with the primary key of another table to maintain data consistency.
MySQL:
An open-source relational database management system used to store and manage structured data.

15. APPENDIX / SQL CODE

The following SQL code was executed using MySQL to implement the Database Normalization Case Study.

```
CREATE DATABASE NormalizationDB;
USE NormalizationDB;

CREATE TABLE Employee_Unnormalized (
    emp_id INT,
    emp_name VARCHAR(100),
    department VARCHAR(100),
    project_name VARCHAR(150),
    manager_name VARCHAR(100),
    skills VARCHAR(200)
);

INSERT INTO Employee_Unnormalized VALUES
(1, 'Amit', 'IT', 'Website Development', 'Rahul', 'SQL, Python'),
(1, 'Amit', 'IT', 'App Development', 'Rahul', 'SQL, Python'),
(2, 'Priya', 'HR', 'Recruitment System', 'Suman', 'Communication, Excel');

CREATE TABLE Employees_1NF (
    emp_id INT,
    emp_name VARCHAR(100),
    department VARCHAR(100),
    project_name VARCHAR(150),
    manager_name VARCHAR(100),
    skill VARCHAR(100)
);

CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100),
    department VARCHAR(100)
);

CREATE TABLE Projects (
    proj_id INT PRIMARY KEY,
    project_name VARCHAR(150)
);

CREATE TABLE EmployeeProjects (
    emp_id INT,
    proj_id INT,
    PRIMARY KEY (emp_id, proj_id),
    FOREIGN KEY (emp_id) REFERENCES Employees(emp_id),
    FOREIGN KEY (proj_id) REFERENCES Projects(proj_id)
);
```

```
CREATE TABLE Managers (
   mgr_id INT PRIMARY KEY,
   manager_name VARCHAR(100)
);

CREATE TABLE Projects_3NF (
   proj_id INT PRIMARY KEY,
   project_name VARCHAR(150),
   mgr_id INT,
   FOREIGN KEY (mgr_id) REFERENCES Managers(mgr_id)
);

SELECT e.emp_name, p.project_name, m.manager_name
FROM Employees e
JOIN EmployeeProjects ep ON e.emp_id = ep.emp_id
JOIN Projects_3NF p ON ep.proj_id = p.proj_id
JOIN Managers m ON p.mgr_id = m.mgr_id;

SHOW TABLES;
```